



تمرینات گیت و پایتون هفته دوم

پروژه جامع کاروند: مدیریت اطلاعات کاروندها با Python ، Git و JSON

توضیح کلی پروژه

در این پروژه باید یک برنامه پایتونی برای مدیریت اطلاعات کاروندهای بوت کمپ طراحی کنید. دقت داشته باشید هر جایی که خروجی را به صورت اسکرین شات از شما درخواست کرد می بایست خروجی دستور گیت را اسکرین شات ذخیره کنید و در هنگام تحویل پروژه چرایی خروجی را توضیح دهید.

هدف پروژه این است که شما بتوانید همزمان سه مهارت را به صورت یکپارچه تمرین کنید:

۱. برنامه نویسی با Python

۲. ذخیره و مدیریت داده ها با JSON

۳. مدیریت نسخه های پروژه با Git و GitHub

نام رپازیتوری پروژه باید به شکل زیر باشد:

karvand-manager

در صورتی که در تمرین کلاسی با این نام ایجاد کردید با همان ریپو ادامه دهید. و طبیعتاً بخش اول که ساخت پروژه است برای شما متفاوت است و نیازی به ساخت و کلون ندارید و در صورتی که از قبل کانفیگ را انجام دادین از این مرحله عبور کنید.

سناریوی پروژه

فرض کنید مسئول بوت کمپ کاروند می خواهید اطلاعات کاروندهای ثبت نام شده را در یک برنامه ساده مدیریت کند.

هر کاروند اطلاعاتی مانند نام، ایمیل، شهر، تحصیلات و مهارت ها دارد. برنامه باید بتواند اطلاعات کاروندها را ذخیره کند، نمایش دهد، جست و جو کند، ویرایش کند، حذف کند و در پایان یک گزارش کلی از وضعیت کاروندها تولید کند.

اطلاعات برنامه باید در فایل JSON زیر ذخیره شود:

نکته مهم: فایل JSON نباید به صورت دستی پر شود. برنامه باید خودش این فایل را بسازد و اطلاعات باید فقط از طریق اجرای برنامه وارد، ویرایش یا حذف شوند.

بخش اول: ساخت ریپازیتوری و آماده سازی Git

ابتدا در GitHub یک ریپازیتوری جدید بسازید.

مشخصات ریپازیتوری:

Repository name: karvand-manager

هنگام ساخت ریپازیتوری، گزینه مربوط به ساخت README را فعال نکنید.

سپس ریپازیتوری را با لینک HTTPS روی سیستم خود clone کنید:

```
git clone https://github.com/USERNAME/karvand-manager.git
```

به جای USERNAME نام کاربری GitHub خودتان را قرار دهید.

وارد پوشه پروژه شوید:

```
cd karvand-manager
```

نام و ایمیل Git را فقط برای همین پروژه تنظیم کنید:

```
git config user.name "Your Name"
```

```
git config user.email "your-email@example.com"
```

برای بررسی تنظیمات Git دستور زیر را اجرا کنید:

`git config --list`

یک فایل به نام README.md بسازید و داخل آن بنویسید:

`# Karvand Manager`

This repository is for the final Python, JSON and Git project of Karvand bootcamp.

اولین commit پروژه را ثبت کنید:

`git add README.md`

`git commit -m "docs: add initial README"`

نام branch اصلی را روی main تنظیم کنید:

`git branch -M main`

پروژه را به GitHub ارسال کنید:

`git push -u origin main`

حالا برای انجام پروژه اصلی، یک branch جدید بسازید:

`git switch -c final-project`

بخش دوم: ساختار فایل ها و پوشه ها

```
karvand-manager/  
├─ app.py  
├─ README.md  
├─ .gitignore  
└─ data/  
    ├─ karvands.json  
    └─ report.json
```

ساختار پروژه باید در پایان به شکل زیر باشد:

فایل اصلی برنامه باید app.py باشد.

فایل karvands.json برای ذخیره اطلاعات کاروندها استفاده می‌شود.

فایل report.json برای ذخیره گزارش نهایی برنامه استفاده می‌شود. این قسمت اختیاری است که در ادامه با آن آشنا

میشوید

بخش سوم: ساختار داده JSON

ساختار کلی فایل data/karvands.json باید شبیه نمونه زیر باشد:

```
{
  "bootcamp": {
    "title": "Karvand Python",
    "year": 2026
  },
  "karvands": [
    {
      "id": 1,
      "full_name": "Ali Ahmadi",
      "email": "ali@example.com",
      "city": "Tehran",
      "education": {
        "degree": "Bachelor",
        "field": "Computer Engineering"
      },
      "skills": [
        {
          "name": "Python",
          "level": "beginner",
          "score": 70
        }
      ]
    }
  ]
}
```

این نمونه فقط برای فهم ساختار داده است. شما نباید این اطلاعات را دستی داخل فایل JSON بنویسید. برنامه باید خودش فایل را بسازد و داده‌ها را در آن ذخیره کند.

در این ساختار:

- bootcamp اطلاعات کلی دوره را نگه می‌دارد.
- karvands یک لیست از کاروندهاست.
- هر کاروند یک id یکتا دارد.
- اطلاعات تحصیلی داخل education ذخیره می‌شود.

- مهارت‌ها داخل skills ذخیره می‌شوند.
 - مقدار skills باید حتماً لیست باشد، چون هر کاروند می‌تواند چند مهارت داشته باشد.
-

بخش چهارم: امکانات برنامه

برنامه باید یک منوی متنی داشته باشد و تا زمانی که کاربر گزینه خروج را انتخاب نکرده است، اجرا بماند.

منوی برنامه باید شامل گزینه‌های زیر باشد:

- ۱) افزودن کاروند
 - ۲) نمایش همه کاروندها
 - ۳) جست‌وجوی کاروند با id
 - ۴) جست‌وجوی کاروند بر اساس مهارت (این قسمت اختیاری)
 - ۵) ویرایش اطلاعات کاروند
 - ۶) حذف کاروند
 - ۷) گزارش کلی (این قسمت اختیاری)
 - ۸) خروج
-

بخش پنجم: توضیح دقیق امکانات برنامه

۱) افزودن کاروند

برنامه باید اطلاعات زیر را از کاربر دریافت کند:

- نام کامل
- ایمیل

- شهر
- مدرک تحصیلی
- رشته تحصیلی
- نام مهارت
- سطح مهارت
- امتیاز مهارت

برای هر کاروند باید یک id عددی و یکتا ساخته شود.

امتیاز مهارت باید عددی بین ۰ تا ۱۰۰ باشد. اگر کاربر مقدار غیر عددی وارد کرد یا عددی خارج از این بازه وارد کرد، برنامه نباید متوقف شود و باید پیام مناسب نمایش دهد.

2) نمایش همه کاروندها

برنامه باید همه کاروندهای ثبت شده را از فایل JSON بخواند و نمایش دهد.

برای هر کاروند، حداقل اطلاعات زیر نمایش داده شود:

- id
- نام کامل
- ایمیل
- شهر
- تحصیلات
- مهارتها

اگر هیچ کاروندی ثبت نشده بود، برنامه باید پیام مناسب نمایش دهد.

3 جست و جوی کاروند با id

برنامه باید از کاربر یک id بگیرد و داخل فایل data/karvands.json جست و جو کند.

اگر کاروند پیدا شد، اطلاعات کامل او نمایش داده شود.

اگر کاروند پیدا نشد، پیام زیر یا پیام مشابه نمایش داده شود:

کاروندی با این شناسه پیدا نشد.

4 جست و جوی کاروند بر اساس مهارت (این قسمت اختیاری است و انجام آن نمره اضافی دارد)

برنامه باید از کاربر نام یک مهارت را بگیرد.

مثلاً:

Python

سپس همه کاروندهایی را نمایش دهد که آن مهارت را دارند.

جست و جو باید روی مقدار name داخل لیست skills انجام شود.

اگر هیچ کاروندی با آن مهارت پیدا نشد، برنامه باید پیام مناسب نمایش دهد.

5 ویرایش اطلاعات کاروند

برنامه باید از کاربر id کاروند را بگیرد.

اگر کاروند وجود داشت، کاربر بتواند حداقل یکی از اطلاعات زیر را ویرایش کند:

- ایمیل
- شهر
- مدرک تحصیلی
- رشته تحصیلی

بعد از ویرایش، فایل JSON باید به روزرسانی شود.

اگر کاروند با آن id وجود نداشت، برنامه نباید خطا بدهد و باید پیام مناسب نمایش دهد.

6) حذف کاروند

برنامه باید از کاربر id کاروند را بگیرد.

اگر کاروند وجود داشت، از لیست karvands حذف شود و فایل JSON دوباره ذخیره شود.

اگر کاروند وجود نداشت، برنامه باید پیام مناسب نمایش دهد.

7) گزارش کلی (مرحله امتیازی) این بخش از سوال اختیاری است و در صورت انجام نمره اضافی دارد

برنامه باید یک گزارش کلی از اطلاعات ذخیره شده تولید کند.

گزارش باید شامل موارد زیر باشد:

- تعداد کل کاروندها
- تعداد کل مهارت‌های ثبت شده
- میانگین امتیاز مهارت‌ها
- فهرست شهرهای ثبت شده

- فهرست مهارت‌های غیرتکراری

گزارش باید هم در خروجی برنامه نمایش داده شود و هم در فایل زیر ذخیره شود:

data/report.json

نمونه ساختار فایل گزارش:

```
{  
  "total_karvands": 3,  
  "total_skills": 3,  
  "average_skill_score": 78.5,  
  "cities": ["Tehran", "Karaj"],  
  "unique_skills": ["Python", "Git", "JSON"]  
}
```

بخش ششم: مدیریت خطاها

برنامه باید حالت‌های زیر را کنترل کند:

۱. اگر پوشه data وجود نداشت، برنامه خودش آن را بسازد.
 ۲. اگر فایل data/karvands.json وجود نداشت، برنامه خودش آن را با ساختار اولیه بسازد.
 ۳. اگر لیست کاروندها خالی بود، برنامه پیام مناسب نمایش دهد.
 ۴. اگر کاربر id ناموجود وارد کرد، برنامه خطا ندهد.
 ۵. اگر کاربر به جای عدد، متن وارد کرد، برنامه متوقف نشود.
 ۶. اگر امتیاز مهارت کمتر از ۰ یا بیشتر از ۱۰۰ بود، دوباره مقدار درست از کاربر گرفته شود.
 ۷. اگر فایل JSON خالی یا خراب بود، برنامه پیام مناسب نمایش دهد یا فایل را با ساختار اولیه بازسازی کند.
-

بخش هفتم: محدودیت‌های فنی

در این پروژه فقط مجاز هستید از کتابخانه‌های استاندارد Python استفاده کنید.

استفاده از کتابخانه‌های زیر مجاز است:

json

os

بخش هشتم: مراحل انجام پروژه با Git

در این پروژه فقط خروجی نهایی مهم نیست. روند انجام کار با Git هم بخشی از نمره پروژه است.

مرحله ۱: بررسی وضعیت اولیه پروژه

قبل از شروع کدنویسی مطمئن شوید روی branch پروژه هستید:

git branch

git status -sb

باید روی branch زیر باشید:

final-project

(از خروجی سه دستور بالا اسکرین شات داشته باشید)

مرحله ۲ Commit: اول؛ ساخت فایل اصلی و ساختار اولیه JSON

در این مرحله فایل app.py را بسازید.

برنامه باید بتواند در صورت نبودن پوشه data و فایل data/karvands.json، آن‌ها را بسازد.

بعد از انجام این بخش، وضعیت پروژه را بررسی کنید:

```
git status -sb
```

```
git diff
```

(از خروجی دستور بالا اسکرین شات داشته باشید)

سپس تغییرات را commit کنید:

```
git add app.py
```

```
git commit -m "feat: create initial json storage"
```

مرحله ۳ Commit: دوم؛ افزودن و نمایش کاروندها

در این مرحله امکانات زیر را کامل کنید:

- افزودن کاروند
- نمایش همه کاروندها
- ذخیره داده‌ها در data/karvands.json

برنامه را اجرا کنید و حداقل دو کاروند از طریق خود برنامه اضافه کنید.

سپس تغییرات را بررسی و commit کنید:

```
git status -sb
```

```
git diff
```

```
git add app.py data/karvands.json
```

```
git commit -m "feat: add and list karvands"
```

(از ترمینال عکس داشته باشید الزامی به ۴ عکس نیست یک عکس از ترمینال داشته باشید که شامل خروجی های بالا

باشد کافی است)

مرحله ۴: استفاده از git stash

در این مرحله شروع کنید به اضافه کردن قابلیت جست‌وجوی کاروند با id.

مثلاً تابع زیر را به‌صورت ناقص به برنامه اضافه کنید:

```
def search_karvand_by_id():  
    pass
```

چون کد هنوز کامل نیست، نباید commit بزنید.

ابتدا تغییرات را ببینید:

```
git status -sb  
git diff
```

حالا تغییرات نیمه‌کاره را موقتاً کنار بگذارید:

```
git stash push -m "start search karvand by id"
```

وضعیت پروژه و لیست stash ها را بررسی کنید:

```
git status -sb  
git stash list
```

(اسکرین شات از دستورات بالا لطفا داشته باشید)

در این مرحله برنامه سالم قبلی را اجرا کنید:

```
python app.py
```

نکته مهم: در این مرحله فقط منوی برنامه را بررسی کنید و داده جدیدی وارد نکنید.

حالا تغییرات نیمه‌کاره را برگردانید:

git stash pop

git status -sb

(اسکرین شات بگیرید لطفاً از خروجی)

بعد از برگشت تغییرات، قابلیت جست‌وجوی کاروند با id را کامل کنید.

مرحله ۵ Commit سوم: جست‌وجو

در این مرحله امکانات زیر را کامل کنید:

- جست‌وجوی کاروند با id
- جست‌وجوی کاروند بر اساس مهارت (اختیاری)

برنامه را اجرا کنید و جست‌وجو را حداقل در این سه حالت تست کنید:

۱. جست‌وجو با یک id موجود

۲. جست‌وجو با یک id ناموجود

۳. جست‌وجو با یک مهارت موجود

سپس تغییرات را commit کنید:

git add app.py

git commit -m "feat: add karvand search features"

مرحله ۶ Commit چهارم: ویرایش، حذف و گزارش (چون گزارش اختیاری است طبیعتاً داشتن report

نیز اختیاری می باشد)

در این مرحله امکانات زیر را کامل کنید:

- ویرایش اطلاعات کاروند
- حذف کاروند
- تولید گزارش کلی
- ذخیره گزارش در data/report.json

برنامه را اجرا کنید و همه امکانات این مرحله را تست کنید.

سپس تغییرات را commit کنید:

```
git add app.py data/karvands.json data/report.json
git commit -m "feat: add edit delete and report"
```

مرحله ۷: ساخت فایل .gitignore

یک فایل به نام .gitignore بسازید.

داخل آن فقط این مورد را بنویسید:

debug.log

حالا یک فایل آزمایشی به نام debug.log بسازید و داخل آن یک متن ساده بنویسید:

Program started...

وضعیت Git را بررسی کنید:

git status -sb

فایل debug.log نباید در لیست فایل‌های Git دیده شود.

فقط فایل .gitignore را commit کنید:

```
git add .gitignore  
git commit -m "chore: ignore debug log"
```

در پایان، اگر فایل `debug.log` هنوز در پوشه پروژه وجود دارد، آن را حذف کنید.

مرحله ۸: ارسال branch پروژه به GitHub

branch پروژه را به GitHub ارسال کنید:

```
git push -u origin final-project
```

مرحله ۹: ادغام پروژه با main

بعد از کامل شدن پروژه، branch پروژه را با main ادغام کنید.

ابتدا به branch اصلی برگردید:

```
git switch main
```

آخرین تغییرات main را دریافت کنید:

```
git pull
```

حالا branch پروژه را merge کنید:

```
git merge final-project
```

تغییرات نهایی را به GitHub ارسال کنید:

```
git push origin main
```

برای بررسی تاریخچه پروژه، دستور زیر را اجرا کنید:

git log --online --graph --all

خروجی از دستور بالا را لطفا داشته باشید

بخش نهم: خروجی‌های مورد انتظار برای تحویل

برای تحویل پروژه، موارد زیر را ارسال کنید:

۱. لینک ریپازیتوری GitHub

۲. فایل app.py

۳. فایل data/karvands.json

۴. فایل data/report.json (در صورت وجود)

بخش دهم: سؤال تشریحی

در پایان پروژه، سوال‌های زیر از شما شفاهی پرسیده میشود نیازی به نوشتن نیست فقط روی آن فکر کنید.

۱. چرا اطلاعات برنامه را داخل فایل JSON ذخیره کردیم؟

۲. چرا برای قابلیت‌های مختلف پروژه چند commit جداگانه ثبت کردیم؟

۳. git stash در این پروژه چه کاربردی داشت؟

۴. چرا فایل debug.log را وارد Git نکردیم؟

۵. تفاوت git push و git pull در روند این پروژه چیست؟

۶. Merge conflict در Git چه زمانی به وجود می‌آید؟ یک مثال ساده از همین پروژه بنویسید.